



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Engineering, Built Environment and IT
Department of Computer Science

Imperative Programming
COS 132
Examination

10 July 2020 (16:00 to 18:00)

Examiners

Internal: Dr Linda Marshall, Dr Patricia Lutu, Mr Emilio Singh, Ms Ntombi Banda, Mr Pula Rammoko, Ms Tari Mautsa and Mr Theo Naidu

External: Dr Christopher Cleghorn

Instructions

1. Read the question paper carefully and answer all the questions.
2. The assessment opportunity comprises of **5** questions on **8** pages.
3. **2** hours have been allocated for you to complete this paper. An additional 30 min have been allocated for you to download the paper and upload your answer document.
4. This paper is **take home** and is subject to the University of Pretoria Integrity statement provided below.
5. You are allowed to consult any literature.
6. You are not allowed to discuss the questions with anyone.
7. If you have any queries when writing the paper, post them in good time on the ClickUP Discussion Board under the appropriate thread, by replying to the thread, in the **Examination** Forum. Note, this forum is moderated and therefore your post will not be available for viewing to the COS132 class until it has been moderated. NO emails during the test will receive a response, only discussion board posts will be responded to.
8. Write your answers in a separate document (eg. using a word processor or handwritten and scanned) and submit the document in **PDF format**. An upload slot will be open on the module ClickUP page under the **Examination - Question paper and upload** menu option for the duration of the examination (16:00 to 18:00) and then for an additional 30 min to give enough time to download this paper and upload your PDF. **No late submissions will be accepted.**
 - Please make sure your **name and student number is clearly visible** in the document you upload.
 - Additionally, provide an e-mail address and a phone number on your paper where you can be contacted, should there be any problems with your document.
9. **Marks per question**

Question:	1	2	3	4	5	Total
Marks:	14	22	9	3	16	64

Full marks is: 60

Integrity statement:

The University of Pretoria commits itself to produce academic work of integrity. I affirm that I am aware of and have read the Rules and Policies of the University, more specifically the Disciplinary Procedure and the Tests and Examinations Rules, which prohibit any unethical, dishonest or improper conduct during tests, assignments, examinations and/or any other forms of assessment. I am aware that no student or any other person may assist or attempt to assist another student, or obtain help, or attempt to obtain help from another student or any other person during tests, assessments, assignments, examinations and/or any other forms of assessment.

Multiple choice questions

1. For each of the questions that follow, choose the most appropriate option.

(a) What does the following loop calculate?

(1)

```
int product = 1;
for ( int i = 3; i <= 15; i += 2 )
    product *= i;
```

- A. The product of the odd integers from 1 to 15.
- B. The product of the even integers from 1 to 15
- C. The product of the odd integers from 3 to 15
- D. The product of the even integers from 3 to 15

(b) Which of the switch-statements is equivalent to the following nested if-statement?

(1)

```
if ( choice == 1 ) {
    cout<<"choice 1 selected: " <<value<<endl;
} else if ( choice == 2 ) {
    cout<<"choice 2 selected: " <<value+2<<endl;
} else if ( choice == 3 || choice == 4 ) {
    cout<<"choice 3 or 4 selected: " <<value+7<<endl;
} else {
    cout<<"default selected: " <<value<<endl;
}
```

A.

```
switch( choice ){
    case 1: cout<<"choice 1 selected: "<<value<<endl;
            break;
    case 2: cout<<"choice 2 selected: "<<value+2<<endl;
            break;
    case 3: cout<<"choice 3 or 4 selected: "<<value+7<<endl;
            break;
    case 4: cout<<"choice 3 or 4 selected: "<<value+7<<endl;
            break;
    default: cout<<"default selected: "<<value<<endl;
}
```

B.

```
switch( choice ){
    case 1: cout<<"choice 1 selected: "<<value<<endl;
            break;
    case 2: cout<<"choice 2 selected: "<<value+2<<endl;
            break;
    case 3:
    case 4: cout<<"choice 3 or 4 selected: "<<value+7<<endl;
            break;
    case 5: cout<<"default selected: "<<value<<endl;
}
```

C.

```
switch(choice){
    case 1: cout<<"choice 1 selected: "<<value<<endl;
            break;
    case 2: cout<<"choice 2 selected: "<<value+2<<endl;
            break;
    case 3:
    case 4: cout<<"choice 3 or 4 selected: "<<value+7<<endl;
            break;
    default: cout<<"default selected: "<<value<<endl;
}
```

(c) What will the output of the following code be? (1)

```
int a[] = {5,6,7,8};
int *b = new int [5];
int *c = b;
```

```
if (a != c)
    cout << "The arrays are not the same.";
cout << "The arrays are the same.";
```

- A. A compilation error.
- B. The arrays are the same.
- C. The arrays are not the same. The arrays are the same.
- D. The arrays are not the same. The arrays are the same.
- E. The arrays are not the same.

(d) Suppose you had to search for the value **10** in an array with elements {2, 4, 6, 8, 10, 12} using the binary search algorithm. What would the **middle** variable contain in the first iteration of the algorithm? (1)

- A. 8
- B. 6
- C. 2
- D. 3

(e) Why is it true that selection sort, usually, performs fewer exchanges of elements in an array as compared to bubble sort? (1)

- A. The elements to be moved are placed at their final sorted position in the array while moving.
- B. The overall complexity of selection sort results in fewer overall sorts being performed.
- C. Bubble sort relies on an unnecessary final pass through the array.
- D. Bubble sort will shift elements even if they are in the correct final place in the array.

(f) What is the most important necessary condition for a binary search algorithm to be applied on an array? (1)

- A. The array cannot be larger than 150 elements.
- B. The array has to be sorted.
- C. The array has to be zero initialised before searching.
- D. The array must have an odd length.

(g) At most, how many comparisons will be needed for a binary search algorithm to search through an array of 15000 elements? (1)

- A. 2^{14}
- B. 2^7
- C. 2^{20}
- D. 2^{15000}

(h) What will **strlen** on **st1** return after the following code executes? (1)

```
const int size =20;
char st1[size] = "Hostess_Snack_";
char st2[] = "Cakes";
strcat(st1 ,st2);
```

- A. 20
 - B. 5
 - C. 19
 - D. 12
- (i) When are the member variables of a struct actually created in memory? (1)
- A. When the struct is declared.
 - B. When the struct is first used.
 - C. When the struct is accessed.
 - D. When the struct variable is defined.
- (j) Can you initialise the variables of a struct using the curly bracket method as you might an array when it is first defined? (1)
- A. Yes
 - B. No
- (k) What is the default access for members of the struct and class abstract data types, respectively? (1)
- A. private for struct, public for class
 - B. public for struct, private for class
 - C. public for struct, public for class
 - D. private for struct, private for class
- (l) What term is used to refer to a class member function that solely retrieves a value from a class's member variable without modifying it? (1)
- A. accessor
 - B. overloading
 - C. mutator
 - D. encapsulation
- (m) What term is used to refer to a class member function that solely stores or changes a value of a class's member variable? (1)
- A. accessor
 - B. overloading
 - C. mutator
 - D. encapsulation
- (n) Which of the following class function prototypes ensures that the `getPrice` function does not modify any of its class's member variables? (1)
- A. `const double getPrice();`
 - B. `double const getPrice();`
 - C. `double getPrice() const;`
 - D. `double getPrice(); const`

Short questions

2. For each of the questions that follow, fill in the blanks. Blanks are denoted by `--[N.]--`, where N is an upper-case Roman numeral.

- (a) Complete the following code so that the loop executes 10 times (3)

```

int j = --[I.]--;
int i = 1;

while ( --[II.]-- )
{
    i = i + --[III.]--;
}

```

- (b) Complete the following function which sorts C-strings into alphabetic order. (4)


```

void bubbleSort( char names[ ][COLS], int count)
{
    int pass, index;
    char temp[COLS];
    for ( pass = 0; pass < count; pass++ )
        for ( index = 0; index < (count -1); index++ )
            if ( --[IV.]-- )
            {
                //swap the names
                --[V.]--
                --[VI.]--
                --[VII.]--
            }
}

```

- (c) You are given the following incomplete program to test whether a given password is valid or not. The rules for a valid password are given by: (6)

1. A password must contain at least one lowercase letter, one uppercase letter and one decimal digit
2. The password length must be at least 7 characters.

```

int main()
{
    char* password;
    password = new char[250];

    // Read the password
    cin >> password;

    //Test whether the password is valid
    if (isValidPassword(password))
    {
        cout << "The password complies with the rules" << endl;
    }
    else
    {
        cout << "The password does not comply with the rules" << endl;
    }

    delete [] password;

    return 0;
}

```

Complete the function to test for a valid password.

```

bool isValidPassword( const --[VIII.]-- )
{
    const int MINLEN = 7;
    int lowerCount = 0 , upperCount = 0 , digitCount = 0, index = 0;
    bool valid = false ;

    --[IX.]-- (password[index] != '\0')
    {
        if ( --[X.]-- (password[index]) )
            digitCount++;
        else if ( --[XI.]-- (password[index]) )
            lowerCount++;
        else if ( --[XII.]-- (password[index]) )
            upperCount++;

        index++;
    }
}

```

```

        if ((digitCount > 0) && (lowerCount > 0) && (upperCount > 0) &&
            (strlen(password) >= MINLEN))
            valid = true;
    }
    --[XIII.]--
}

```

(d) Complete the following struct definition. (1)

```

struct Bus{
    --[XIV.]-- routeName;
    int routeLength;
};

```

(e) Given a struct (2)

```

struct Cart {
    string material;
    double volume;
    int cap[3];
};

```

and the following code

```

int main()
{
    Cart c = --[XV.]--;
}

```

Instantiate **c** with the following values:

- tin
- 70.3
- {1,2,3}

(f) Use the following prompts to define a class **Cube** that contains information about a cube's length and volume. (3)

i. Complete the declaration for the **Cube** class by filling in the missing code statements. The class should contain a public interface to modify and retrieve the length of the cube and to compute its volume. (3)

```

class Cube
{
    --[XVI]--          // access specifier
    int length;

    --[XVII]--         // access specifier
    --[XVIII]--        // setLength function prototype
    --[XIX]--          // getLength function prototype
    --[XX]--           // getVolume function prototype
};

```

ii. Implement the **getVolume** function to return the cube's volume as follows: $V = l^3$, where V and l are the cube's length and volume, respectively. (2)

```

int --[XXI]--
{
    return --[XXII]--
}

```

iii. Why is it advisable to not have a variable **volume** to store the cube's volume value? (1)

3. For each of the following questions, determine the output of the program or code segment. (1)

(a) Consider the code below. (1)

```

char inChar;
cin >> inChar
while ((inChar == 'H') && (inChar == 'h'))
{

```

```

    if (inChar != 'H')
    {
        cout << '*';
    }
    else
        cout << "**";
    cin >> inChar;
}
cout << "finished";

```

What output will the loop below produce if the user keys in **h**?

- (b) What is the output of the following code segment if **value** is the character %? (3)

```

i = 0;
while (i < 7)
{
    int j = 7 - i;
    while (j > 0) {
        cout << value ;
        j--;
    }
    cout<<endl ;
    i ++;
}

```

- (c) You are given the following list to sort:

5	3	8	2
---	---	---	---

 . What are the contents of the list after the **first** (2)
pass or scan of the following algorithms:

- i. bubble sort algorithm
- ii. selection sort algorithm

- (d) Consider the following struct declaration and main function. (3)

```

struct TeaStand
{
    double price;
    int* numTeasLeft;
    int* weekSales;
};

int main()
{
    TeaStand tea = {5.5};
    TeaStand* ptr = &tea;
}

```

Assume the following initialisations for the **numTeasLeft** and **weekSales** members of the **tea** struct variable:

- **numTeasLeft** points to an integer with a value of 100
- **weekSales** points to an array with values {20, 40, 35, 45, 55, 30, 50}.

Write down the outputs of the following **cout** statements.

- i. `cout << tea.price << endl;`
- ii. `cout << *ptr.numTeasLeft << endl;`
- iii. `cout << *ptr->numTeasLeft << endl;`
- iv. `cout << *tea.weekSales << endl;`
- v. `cout << *((*ptr).weekSales + 2) << endl;`
- vi. `cout << *tea.numTeasLeft - ptr->weekSales[6] << endl;`

4. Each of the following code segments contain errors. Assuming that all the necessary header files for the functions called have been included in the program, state the error in each of the code segments. (3)

- (a) `char charVar = "B";`
`if ( isupper(charVar) )`
`cout << "Upper case";`

- (b) `char numeric [5];`
`int x = 5;`
`numeric = atoi(x);`
- (c) `char stringA [] = "Kathy";`
`char stringB [] = "Billy Jones";`
`strcat(stringA, stringB);`

Programming question

5. In this question you are going to provide the implementation for a programme. Pay attention to C++ implementation specifics such as headers, and syntax as that will be assessed alongside program logic.

- (a) Provide the definition for a struct called `Bill` with the following attributes: (2)
- A string variable, `name`.
 - An integer variable, `processOrder`.
 - A double variable, `amountDue`.

- (b) Given the following definition of a function `orderBill`, (4)
- ```
void orderBill(Bill *clientBills, int numClients);
```

Implement the function to re-order the client bills in `clientBills` by the `processOrder` attribute in descending order.

- (c) Implement a main program that makes use of the prior struct and function to read in a list of clients from a file called `clients.txt`, and to thereafter display all of the client bills in the order specified by the `orderBill` function. You may assume the struct and function will be included in the same file as the main program and do not need to be specified. (10)

The input file, `clients.txt` has the following format:

```
John,1,11.1
Sipho,3,13.3
Manny,2,12.2
```

The file will have all of the data with one line per client with three variables, the first being the the client `Name`, then the `processOrder` and finally the `amountDue`.

An example of the program output is given below. Use this to orient the implementation's formatting requirements:

```
3,Sipho,13.3
2,Manny,12.2
1,John,11.1
```

Each line of output should have a newline at the end. In your program, you are only allowed to use the following libraries: `iostream`, `fstream`, `string`, `sstream`. You may assume the namespace `std` has been used.